PART 2 · PRODUCT AVAILABILITY REPLY AGENT

Answering from the Data

Every “where can I buy this?” comment gets a reply drafted from the sales reps' shelf visits, and a person sends it in one click.

Omar Qasem · October 2026

What is in the repository

Live sites	The team console and the feed simulator, running on Netlify (deploy steps in the repository README)
Slides	22 slides with speaker notes, as PDF and PowerPoint
Answer Path Explorer	An interactive page that runs the real search, table query and graph code on the sample data: step through the three paths, click any LangGraph node or edge, test any words
Code and deploy files	The source of both sites, ready to deploy on Netlify, with the tests

01 The short answer

I built a **team console** where every comment under the brand's posts already has a checked reply waiting. The marketing team reads it and clicks **Reply now**.

- **Seconds, not a search.** Each comment is read, searched and drafted in the background as soon as it arrives, so the answer is ready before anyone opens it.
- **Facts only from the data.** Search works out which product and which place the customer means. A plain filter on the reps' data gives the shops and visit dates. The AI only writes the sentence, and a check confirms every shop, date and price in it is in the data.
- **An honest “is it available now?”** Every reply says when the shelf was checked, because the data is a record of visits, not live stock.
- **A person sends every reply.** When a comment is unclear the draft becomes a one-click choice; complaints and other questions go to a person.

02 The problem as I read it

PROBLEM	WHAT IT MEANS	WHAT FIXES IT
Slow	Someone filters the sheet by hand for every question, and the sheet keeps growing.	The reply is drafted before anyone opens the comment.
Hard to read	Comments come in Jordanian Arabic or English, with typos, and “it” usually means the product in the post.	The AI reads the comment; search links each word to a product, shop or area; each post is read once, so “it” has a meaning.

PROBLEM	WHAT IT MEANS	WHAT FIXES IT
Risky to get wrong	A public reply that names the wrong shop, or stock from weeks ago, costs trust.	Shops and dates come only from the data, every reply gives the visit date, and a person sends it.

03 What I built

Two small sites and one backend, deployed on Netlify from two zip files.

Feed simulator

Stands in for Instagram and Facebook. The brand publishes posts with an image and a caption; customers pick a name and comment; replies appear under the comments.

Team console

Every comment with its draft and status, the shops and visit dates the draft used, and “How this draft was made”. Reply now, Edit, pick an option, assign or dismiss. Pages for the data upload, the posts and the system's health.

Backend

12 Netlify Functions, two of them background jobs that may run for 15 minutes; two LangGraph graphs; Netlify Blobs for storage; Groq to read and write; Jina for search by meaning.

Each draft has one of four statuses: `Ready` `Needs a choice` `Needs a person` `No reply needed`. Without any AI key the whole system still runs, with rules to read and templates to write.

04 How it works: three paths

Path 1 · The data goes in (on every upload)

01 **Upload the file** on the console's Data page. The site starts with the sample from the brief: 936 rows, 40 shops, 27 products, visits from 12 to 25 August 2026.

02 **Map the columns** by name or synonym, so a file that says `shop` or `outlet` instead of `store_name` still works.

03 **Clean each row.** Dates are read day first, as written in Jordan (19/08/2026 is 19 August). The status comes from the availability column, or from the quantity when it is missing.

04 **Keep the latest visit** for each shop and SKU, so an old “in stock” never outlives a newer “out of stock”.

05 **Build the catalog:** 129 entries (27 products, 40 shops, 39 areas, 12 cities, 8 categories, 3 regions), each with English and Arabic names and the other ways people write them.

06 **Write every name** as a small document: 436 in all, normalized (hamza forms, taa marbuta, diacritics) and without filler words such as وين or متوفر.

07 **Index each name three ways:** BM25 on the words, a character vector of 512 slots, and a Jina vector of 256 numbers. All 436 names go to Jina in one call.

08 **Save it** as one object in Netlify Blobs: the clean table, the catalog and the index with its vectors. At this size that object is the vector store; BM25 statistics and character vectors are rebuilt in memory when a function starts.

Path 2 · A new post (once per post)

Most comments never name the product: they say وين بلاقيه؟ (where do I find it?). So each post is read once, when it is published. The image goes to the media store and the post to the social store, and a background job runs the **post graph**: `read_post` loads the caption (Groq's model reads text only), `spot_products` asks Groq for the products and pack sizes as JSON, `link_catalog` runs each name through the hybrid search over the 27

products and keeps a link only at confidence 0.72 or more, and `save_context` stores the links with the post. The post text is not added to the index; the post points to catalog entries, and the team can correct the links.

Path 3 · A comment, from the feed to a reply

01 The customer's comment reaches the API, the way Meta's webhook would on a real page.

02 The comment and an empty draft are saved, so the console shows it at once as being drafted.

03 A background job starts. The API answers at once; the job drafts one comment at a time, waits out Groq's per-minute limit when it must, and writes a heartbeat every 20 seconds.

04 The job runs the **ask graph** (section 5).

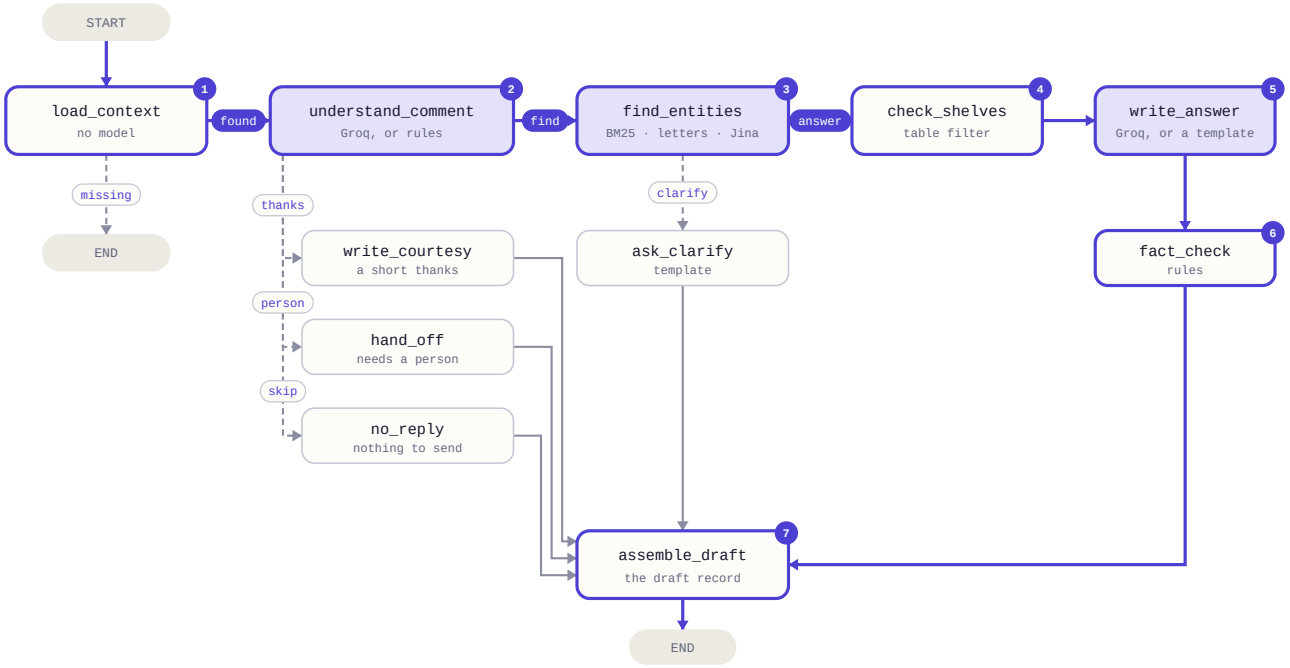
05 The draft is saved with the shops and visit dates it relied on and the trace of every node.

06 The console checks for changes every 3 seconds, so the draft appears within seconds.

07 A person clicks Reply now, edits first, or picks an option when the draft asked for a choice.

08 The reply is saved and appears under the comment on the feed.

05 The LangGraph workflow



The ask graph: 11 nodes, 3 conditional routers (dashed edges), 9 fixed edges. The numbered route is the one the example comment زيت زيتوون بالزرقا متوفر؟ took. Purple nodes call an AI model or the hybrid search.

NODE	MODEL	WHAT IT DOES
`load_context`	none	Loads the comment, its post and the products the post graph linked to it.
`understand_comment`	Groq; rules without a key	Reads the intent, product words, places, shops, a pack size and whether the price is asked, as JSON that must fit a strict schema.
`find_entities`	Jina + BM25 + character vectors	Turns each product and place word into a catalog entry, a choice to ask, or nothing. A category such as زيت takes the post's product; a shop beats an area, an area beats a city.
`check_shelves`	none	Filters the table in the place asked, widening to the city, region and whole country only when nothing is on the shelf (section 7).
`write_answer`	Groq; template without a key	Writes the reply from a small JSON of facts, in Jordanian Arabic or English. The model never sees the sheet.
`fact_check`	none	Every shop, date and price in the reply must be in the facts, and a visit date must be given; otherwise the template replaces the reply.
`ask_clarify`	template	Asks which product or place was meant; the options reach the team as buttons, and a pick runs the graph again.
`write_courtesy`	model's short text or template	A short thank-you. The model's text is used only if it holds no numbers.
`hand_off`	model's short text or template	A holding reply for complaints and other questions; the comment is flagged Needs a person.

NODE	MODEL	WHAT IT DOES
`no_reply`	none	Tags, emojis and spam: nothing to send.
`assemble_draft`	none	Builds the one draft record the console shows.

The three decisions

- **After load_context:** comment found → understand_comment; missing → END.
- **After understand_comment:** availability or price → find_entities; praise → write_courtesy; complaint or other question → hand_off; tag, chat or spam → no_reply.
- **After find_entities:** a choice is needed → ask_clarify; otherwise → check_shelves.

Fixed edges and shared state

Nine fixed edges: START → load_context; check_shelves → write_answer → fact_check → assemble_draft; ask_clarify, write_courtesy, hand_off and no_reply → assemble_draft; assemble_draft → END.

One state with 20 channels; each node returns only what it changes, and the notes and trace are appended. If LangGraph cannot load, the same definition runs on a small built-in runner.

06 How search finds the product and the place

Every one of the 436 names is ranked three ways, and the three rankings are fused.

RANKING	HOW IT IS COMPUTED	WHAT IT CATCHES
BM25 on words	k1 = 1.2, b = 0.75. Each Arabic word is also indexed without its prefixes (ال, ف, ك, ل, ب, و). A word found in fewer names weighs more.	Exact names and rare words; بالزرقا finds الزرقاء.
Character vectors	Every piece of 2, 3 and 4 letters, with # at the word edges, is hashed (FNV-1a) into one of 512 slots; longer pieces weigh more; the vector has length 1.	Typos and spellings: زيتوون against زيتون scores 0.81.
Jina vectors	256 numbers per name from jina-embeddings-v5-text-small; the comment's words are embedded once per search.	Meaning, across languages: olive oil and زيت زيتون share no letters but land close.

Nearest vector. Both kinds of vector have length 1, so the nearest name is the one with the highest dot product, the cosine: $\cos(q, d) = \Sigma\, q_i \times d_i$. Across 436 names that takes a few milliseconds, which is why no separate vector database is needed at this size.

Fusion and decision. Reciprocal rank fusion adds the ranks, not the raw scores, so BM25 points and cosines never need a common scale. A confidence score then decides what happens.

```
score = 1/(60 + BM25 rank) + 1/(60 + character rank) + 1/(60 + Jina rank) · +0.05 for an exact name
```

```
confidence = 1 for an exact name, else 0.55 × word overlap (F1) + 0.45 × character cosine (at least 0.78 when the whole name is in the text)
```

Match at 0.72 or more with no rival of the same type within 0.08; **ask** between 0.55 and 0.72, or with a close rival (رز is both Egyptian and Basmati rice); **no match** below 0.55.

A worked example, with the real numbers

Ahmad comments under the olive oil post: زيت زيتوون بالزرقا متوفر؟ (“Is olive oil available in Zarqa?”, with a typo). The numbers below are the two rankings that need no API, as computed by the system.

Product words: زيت زيتوون

NAME	BM25	LETTERS	RRF	CONF.
زيت · Cooking Oil	#1	#3	0.0323	0.78
زيت زيتون · Olive Oil	#6	#1	0.0315	0.64
زيت قلي · Sunflower	#2	#6	0.0313	0.52

The category wins, so the post's oil is used: Olive Oil. With Groq the typo is fixed first and زيت زيتون is an exact name (1.00).

Place word: بالزرقا, also searched as الزرقا and زرقا

NAME	BM25	LETTERS	RRF	CONF.
الزرقاء · Zarqa city	#1	#1	0.0828	1.00
وسط الزرقاء · area	#3	#3	0.0317	0.71
الزرقاء الجديدة · area	#2	#4	0.0318	0.67

An exact name after normalizing, plus 0.05: a clear match on Zarqa, the city.

check_shelves: olive oil in Zarqa, 4 of 5 shops had it on the shelf at the last visit. The reply names Nour Market Russeifa (24 Aug), Al Amana Grocery Hashemiya (19 Aug) and City Center Zarqa (19 Aug).

07 The answer comes from the table, not the model

Once search has said which product and which place, the answer is a plain filter on the data, so every shop, status, price and date in a reply comes from the reps' visits.

01 Take the rows for the product, and for the pack if one was asked (“2 كيلو”, “the big one”).

02 Keep the place asked: a shop, an area, a city or a region.

03 None on the shelf there? Widen one level at a time: shop → area → city → region → Jordan.

04 Sort: nearest first, in stock before low stock, the newest visit, then more units.

05 Name up to 3 shops with their visit dates; give the shelf price range only when asked.

Example. Emily asks for olive oil near Abdoun. The only shop there that carries it, Sameh Mall Abdoun, was out of stock at the visit on 24 August, so the answer widens to Amman, where 11 of 14 shops had it. The template reply, word for word (emoji removed):

> Hi Emily! Our reps' last visit (24 Aug) didn't find extra virgin olive oil on the shelf in Abdoun, but you can find it in Amman at:
> • Safeway Shmeisani (last visit 22 Aug)
> • Al Rawnaq Grocery Jabal Nasr – Jabal Al Nasr (last visit 20 Aug)
> • Baraka Mini Market Nazzal (last visit 20 Aug)
> Worth checking with the shop before you go

When Groq writes the reply instead, it receives exactly these facts as JSON, and nothing else.

08 Guardrails

A public reply in the brand's name needs more care than an internal task, so five checks run before anything is posted.

CHECK	WHAT IT DOES
Facts only	The model writes from a small JSON of facts for this answer. It never sees the sheet.
Fact check	Every shop, date and price in an AI reply must be in those facts, and the visit date must be given, or the template is used and the trace says why.

CHECK	WHAT IT DOES
Unsure? Ask	Below 0.72, or with two close names, the draft asks instead of guessing; the team picks with one click.
People for people	Complaints and questions the data cannot answer get a holding reply and the status Needs a person.
A person sends it	Nothing is posted until someone clicks Reply now. Edit and Dismiss are always there.

If something fails, it still works. Without a key, after an AI error or at a daily limit, rules read and templates write. If LangGraph cannot load, a built-in runner executes the same graph. Every draft shows how it was made: each node, its note and its time, plus the shops and visit dates it used.

09 Models and methods

JOB	MODEL OR METHOD	SETTINGS	WHY
Read comments, write replies	Groq · openai/gpt-oss-120b	Strict JSON schema, low reasoning effort, up to 1,000 output tokens	Fast, free tier, follows a schema
Search by meaning	Jina · jina-embeddings-v5-text-small	256 numbers, text-matching task	Arabic and English, free key
Search by words	BM25	k1 1.2, b 0.75, Arabic prefix forms	Exact names, rare words
Search by spelling	Character vectors	2 to 4-letter pieces, 512 slots	Typos; no API needed
Fuse the three	Reciprocal rank fusion	score = Σ 1/(60 + rank)	No score scales to tune
Run the steps	LangGraph.js	Two graphs: post and ask	Explicit nodes, edges and state

gpt-oss-120b is OpenAI's open-weight mixture-of-experts model: 117B parameters with 5.1B active per token, 128K context, Apache 2.0. Groq's free tier allows 30 requests and 8,000 tokens a minute for it. jina-embeddings-v5-text-small has 677M parameters, supports 119+ languages, and is trained so its 1,024 numbers can be cut to 256. Gemini or OpenAI can replace either model through one setting.

10 Testing

- **42 of 42** place and product names, in Arabic, English and with typos, resolve to the right catalog entry with no AI at all.
- **22 sample comments** cover dialect, typos, sizes, complaints, tags and English.
- **End to end in a browser**, from the exact zip files that are deployed, in five modes: no key, Groq with Jina, OpenAI, Gemini, and Jina only.
- Each AI service was replaced locally by a stand-in with the same requests, errors and limits, including Groq's 8,000 tokens a minute. The live site runs the real services, and the Health page's **Test the key** button checks both keys.

WHAT THE TESTS CHANGED	BEFORE AND AFTER
رز (rice)	It matched Egyptian rice on its own. Now two names that both hold the word become a choice for the team.
الحجم الوسط (the medium size)	It was read as the Central region. Size words are now removed before places are matched.
Rate limits	Groq's free 8,000 tokens a minute is too little for 18 drafts at once. Now one background job drafts them in turn and waits.

11 Options I weighed

OPTION	WHY NOT	WHAT I DID INSTEAD
Give the AI the whole sheet	936 rows and growing. It can misread a row or invent a shop, and every question pays for the whole sheet.	The table query finds the facts; the AI only writes.
Vector search alone	Embeddings blur exact names and short Arabic words with typos, and everything would depend on one API.	Three rankings fused; two of them need no API.
Reply automatically	These are public replies in the brand's name; trust in the drafts has to be earned first.	Drafts in seconds, a person sends, and the share of drafts sent unedited can be tracked.
n8n again, as in Part 1	This work needs an index, a data table and branching on what the customer meant, not a fixed pipeline.	LangGraph in code, the option I set aside in Part 1.

12 Known gaps and next steps

TODAY	NEXT STEP
The feed is a simulator	Meta's webhooks read the real Instagram and Facebook comments, and the Graph API posts the reply.
Visits in the sample end on 25 August	A daily file from the ERP; the console already warns when the data is over 30 days old.
Groq's free tier: 8,000 tokens a minute	Starting drafts take about 10 minutes after a reset; a paid tier or a smaller reading model removes the wait.
Arabic shop names built from a word list	A one-time review by the team, saved with the data.
Index kept as one object in Netlify Blobs	Right for 436 names; at scale, Postgres with pgvector and the same hybrid search.
Quality is not measured yet	Track how often a draft is edited before sending, per type of comment.

13 How to try it

Open the **team console** and the **feed simulator** side by side. After a reset, the 18 starting drafts take about 10 minutes on Groq's free tier; a new comment is drafted within seconds.

01 On the feed, comment as a customer under the olive oil post, for example وين بلاقيه بإربد؟

02 The draft appears in the console within seconds; open “How this draft was made” to see each node and the shops and dates behind the reply.

03 Click Reply now; the reply shows under the comment on the feed.

04 On وين بلاقيه بماركا؟ وعندكم رز؟ pick Basmati, and the draft becomes a full answer.

05 Publish a new post; its products are linked on their own.

The **Answer Path Explorer** (`Answer-Path-Explorer.html`) runs the same code in a browser, with no setup: step through the three paths, click any node or edge of the graph, and type any words into the search lab to see every BM25 score, character vector, cosine and decision.

To deploy a copy: drag the two zips in `deploy/` onto Netlify (the console first), then add `GROQ_API_KEY` and `JINA_API_KEY` as environment variables in Netlify. Keys never go into the code. Full steps are in README.md.